



北京理工大学
Beijing Institute of Technology

本科生实验报告

课程名称： 电路与电子线路课程设计

实验名称： 数字系统设计

任课教师:	马 越			实验教师:	马 越		
实验日期:	2026 年 1 月 1 日			实验地点:	工训楼 507、506		
实验类型:	☐ 原 理 验 证 ☒ 综 合 设 计 ☐ 自 主 创 新						
学生姓名:	闫子易	班级:	13212302	学号:	1120230621		
学 院:	集成电路与电子学院			专 业:	电子科学与技术		
组 号:	无	同 组 同 学 :		无			
成 绩:							



集成电路与电子学院
SCHOOL OF INTEGRATED CIRCUITS
AND ELECTRONICS

课程设计数字系统 RISC-V 仿真设计

一、实验目的

项目的核心目标是通过从零开始实现一个简洁易懂的 RISC-V 处理器核，让开发者能够在实践中掌握 RISC-V 指令集、Verilog 硬件描述语言以及 FPGA 平台上的处理器设计与调试方法，同时提供一个轻量级、可运行 C 程序、支持中断和外设的完整实验环境，从而加深对嵌入式系统和 CPU 架构的理解

本实验旨在通过对 tinyriscv 开源项目的深入研究与实践，掌握 RISC-V 指令集架构的基本原理及其硬件实现方法。通过分析该精简 RISC-V 处理器核的源代码结构，理解 CPU 核心模块的设计思想，包括取指、译码、执行等关键流水线阶段的工作机制。实验过程中，重点学习如何将计算机体系结构理论知识转化为具体的硬件描述语言实现，掌握处理器设计中的关键技术难点及解决方案。同时，通过搭建仿真环境并运行测试程序，验证处理器功能的正确性，培养硬件系统调试与性能分析能力。最终，通过本实验使学生建立起对现代处理器设计的完整认知体系，为后续开展计算机体系结构相关研究奠定实践基础。

本次课设数字系统我选择 **TASK2** 完成。

1) 将 simple 例程中的 sum 累加功能，移植到 verilog 编写的硬件模块中(例如 timer.v)，从而将软件计算变为硬件计算，cpu 配置并启动硬件模块进行 sum 累加。

2) main.c 中要添加对硬件模块内寄存器值的读取功能，实现硬件模块完成 sum 累加后，通过中断触发 cpu 读取计算结果并进行校验（对应步骤 4）。添加中断功能，实现计算完成后、CPU 回读 sum 结果并校验。

3) GTKwave 软件中可直接观察到 i、sum 这两个寄存器的值，从而看到 sum 循环累加的完整计算过程。

4) timer 的 main.c 中添加对 sum 最终累加值的判断功能，在 cmd 的仿真命令执行后，能看到 pass/fail 提示。

二、实验原理

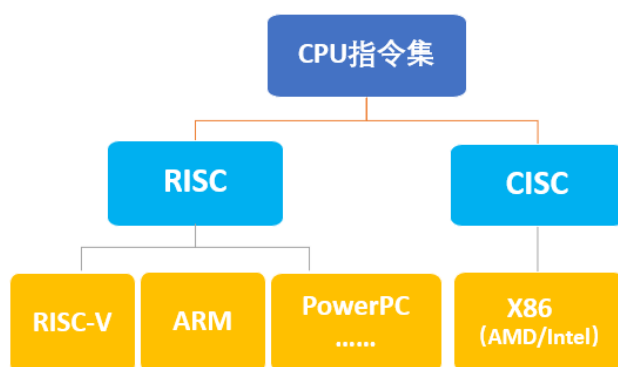
1. 什么是指令集

我们平常说的 X86、ARM 芯片，都是指的芯片的指令集架构。

指令集是软件和硬件之间的接口，简单来说**指令集就是软件与硬件之间沟通的"翻译官"**，是芯片产业中不可或缺的基础部分。



使用不同的指令集，代表着不同的 CPU。目前市面上的 CPU 分类主要分有两大阵营，一个是 Intel、AMD 为首的复杂指令集(CISC)CPU, 另一个是以 IBM、ARM、RISC-V 为首的精简指令集 (RISC) CPU。



不同品牌的 CPU，其产品的架构也不相同，例如，Intel、AMD 的 CPU 是 X86 架构的，而 IBM 公司的 CPU 是 PowerPC 架构，ARM 公司是 ARM 架构。同时，不同的芯片厂商按照指令集的设计情况分为精简指令集和复杂指令集。

那么不同的指令集对 CPU 处理有什么影响呢？

不同的指令集决定着 CPU 的处理方式。从指令集角度来看，CPU 的效率主

要通过两种思路来提升：要不通过降低每个程序所需的指令数来提升效率，要不通过降低每条指令所需的时间周期数来提升效率。

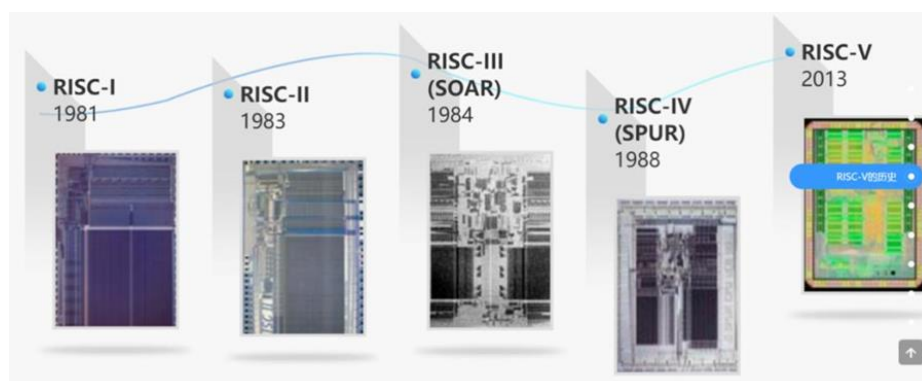
$$\frac{\text{time}}{\text{program}} = \frac{\text{instruction}}{\text{program}} * \frac{\text{cycle}}{\text{instruction}} * \frac{\text{time}}{\text{cycle}}$$

CISC 更偏重前者，而 RISC 更侧重后者。目前来看，CISC 指令集较为复杂，提供了丰富的指令，能够减少程序员的编程工作量。

指令集不仅是软件与硬件之间的接口，更是决定处理器架构特征与性能边界的关键因素。不同的指令集往往体现了设计者在复杂度、扩展性和效率之间的取舍，例如精简指令集强调简洁和高效，而复杂指令集则追求功能的丰富性。实验中我们关注 RISC-V 的意义，正是因为它作为一种开源的精简指令集架构，提供了灵活的扩展能力和透明的规范，使得研究者和产业能够在统一的基础上进行创新与优化。因此，在分析芯片之前，明确指令集的作用和定位是必要的，它不仅决定了硬件的实现方式，也直接影响了软件生态的构建与发展。

2. RISC-V 芯片架构

在理解指令集的作用之后，我们进一步认识到，不同的处理器架构往往体现了设计者对同一功能的不同实现思路。长期以来，X86 与 ARM 是应用最广泛的两类架构，但前者由英特尔和 AMD 严格封闭，后者则需要支付高昂的授权费用，这在一定程度上限制了产业的自由发展。



为突破这一局限，20 世纪 80 年代初，加州大学伯克利分校的 David Patterson 教授与斯坦福大学的 John L. Hennessy 教授开始尝试对传统的复杂指令集进行精简，由此催生了 RISC 的概念，并引发了信息技术史上著名的 CISC 与 RISC 之

争。1980 年，伯克利 RISC 项目设计出第一代 RISC-I 处理器，奠定了后续架构的基础。随后，RISC-II、RISC-III 与 RISC-IV 相继问世，逐步完善了精简指令集的设计理念，并推动了 MIPS、IBM PowerPC 以及 ARM 等新架构的出现。

进入 21 世纪初期，开源运动在操作系统与应用软件领域取得了显著成功，典型代表如 Linux 与 Mozilla，这为开源芯片设计提供了新的契机。2010 年，伯克利科研团队在调研现有的 X86 与 ARM 指令集后，决定重新设计一套全新的指令集。在这样的背景下，RISC-V 作为第五代精简指令集计算机架构正式诞生。其名称既表明了源自伯克利的传承，也通过罗马数字“V”暗示了变种与向量的支持，体现了面向多样化体系结构研究的目标。最初，学术界对 RISC-V 的创新性持怀疑态度，未能在顶级会议上获得广泛关注。但随着伯克利团队将其从概念推进到原型芯片，并在 2015 年成立非盈利组织 RISC-V 基金会，经过多次技术研讨与推广，RISC-V 逐渐走向成熟并获得产业界的认可。

RISC-V 架构最突出的特点首先在于完全开源。RISC-V 指令集的使用不需要支付高昂的授权费用，基金会采用宽松的 BSD 协议，使得企业能够自由地使用和扩展，同时还允许在不公开的情况下添加自有指令，从而实现差异化发展。

（1）完全开源（自由、免费、可控）

开源意味着自由、免费、可控。对于 RISC-V 指令集的使用，RISC-V 基金会不收取高额的授权费。开源采用宽松的 BSD 协议，企业可以完全自由免费使用，同时也允许企业添加自有指令集，而不必开放共享，实现差异化发展。而且不必担心某一天像 X86、ARM 芯片架构一样，很容易被限。

（2）架构简单，没有历史包袱

与 X86 和 ARM 这两大成熟架构相比，RISC-V 架构具备显著的后发优势。计算机架构经过多年演进，更趋于成熟，许多问题已被研究透彻，作为后来者的 RISC-V 架构可以自然而然的加以规避。同时，没有向后兼容的历史包袱。直观地说，X86 和 ARM 架构的指令多达数千条，而 RISC-V 架构仅需 100 余条指令。此外，简洁的架构方案还可有效减少错误发生（毕竟 CPU 流片成本是极高的）。

（3）模块化设计

RISC-V 架构不仅短小精悍，其不同的部分还能以模块化的方式组成在一起，

从而试图通过一套统一的架构满足各种不同的应用场景。用户能够灵活选择不同的模块组合,来实现自己定制化设备的需要,比如针对小面积低功耗嵌入式场景,用户可以选择 RV32IC 组合的指令集,仅使用 MachineMode(机器模式);而高性能应用操作系统场景则可以选择 RV32IMFDC 指令集,使用 MachineMode(机器模式)与 UserMode(用户模式)两种模式。



这种开放性保证了研发的自主可控性,避免了像 X86 或 ARM 那样可能受到限制的风险。其次, RISC-V 的架构设计简洁,没有历史兼容的负担。相比 X86 和 ARM 数千条指令的复杂体系, RISC-V 仅需百余条指令即可覆盖核心功能,这不仅降低了设计复杂度,也减少了出错的可能性,尤其在 CPU 流片成本极高的背景下,这种简洁性显得尤为重要。最后, RISC-V 采用模块化设计理念,不同功能单元可以像积木一样灵活组合,以满足不同应用场景的需求。在低功耗嵌入式系统中,可以选择 RV32IC 指令集,仅运行机器模式;而在高性能操作系统环境下,则可采用 RV32IMFDC 指令集,支持机器模式与用户模式的切换。这种模块化的灵活性使得 RISC-V 能够在统一架构下覆盖广泛的应用范围,体现了其在实验与产业实践中的独特优势。

3. tinyRISC-V 整体架构

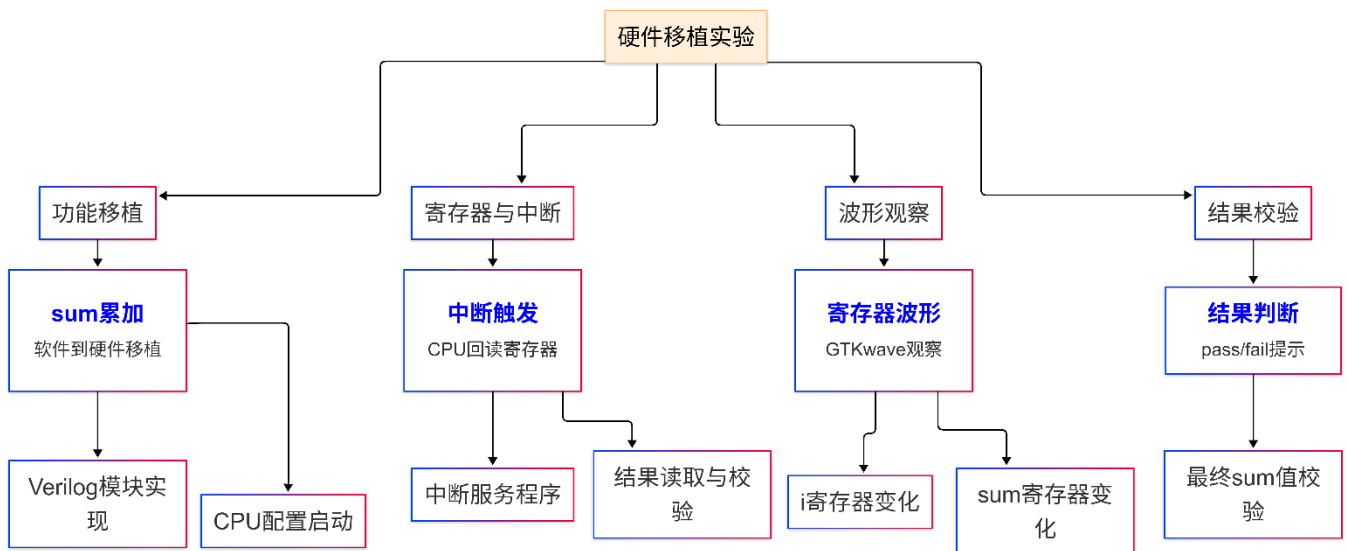
本实验所采用的处理器核心为 tinyriscv,它是一个由 Verilog 语言实现的单核 32 位 RISC-V 处理器。设计目标在于对标 ARM Cortex-M3 系列,以验证开源指令集在嵌入式场景中的可行性与移植性。

三、TASK2 的代码设计

1. 代码整体结构和复现流程（重要!）

我们将 simple 例程中的 sum 累加功能移植到由 Verilog 编写的硬件模块中，以实现由硬件替代软件进行循环累加的计算过程。具体做法是将原本在软件中执行的 sum 运算逻辑嵌入到 timer.v 这样的硬件模块中，由 CPU 进行配置和启动，使得累加操作在硬件电路中自动完成。与此同时，在 main.c 中增加了对硬件模块寄存器的读取功能，当硬件完成累加后会通过中断触发 CPU，CPU 在中断服务程序中回读寄存器中的 sum 结果并进行校验，从而保证硬件计算与预期一致。在仿真过程中，利用 GTKwave 可以直接观察到 i 和 sum 两个寄存器的变化情况，完整呈现了 sum 循环累加的过程和结果。最后，在 timer 的 main.c 中加入了对 sum 最终累加值的判断逻辑，使得在命令行执行仿真命令后能够输出 pass 或 fail 的提示，从而直观地验证硬件累加功能的正确性。这一系列步骤不仅展示了软件功能向硬件实现的移植过程，也通过寄存器读取、中断触发和波形观察等手段完成了对硬件计算结果的验证。

整体结构框图大致如下



对于代码设计，我总体上没有在 gitee 原有 tinyRISCV 项目的基础上增加代码文件，是在 timer_int 这个样例上改写的。所以复现报告中的现象有两种方法，第一种是替换原项目中我修改的文件，第二种是克隆我的 Github 仓库，我已经把所有的完成代码上传到远程仓库，直接克隆下来就能编译运行，比较省事。

(1) 替换原项目的文件

相对于 gitee 原项目我修改了

```
\tests\example\timer_int\main.c
\tests\example\include\timer.h
\rtl\perips\timer.v
\tests\example\include\utils.h
\tb\tinyriscv_soc_tb.v
```

按照对应路径替换上面的文件即可

增加了 TASK1 和 TASK2 的 bin 文件和 vcd 波形文件，可以直接验证题目要求功能是否实现。

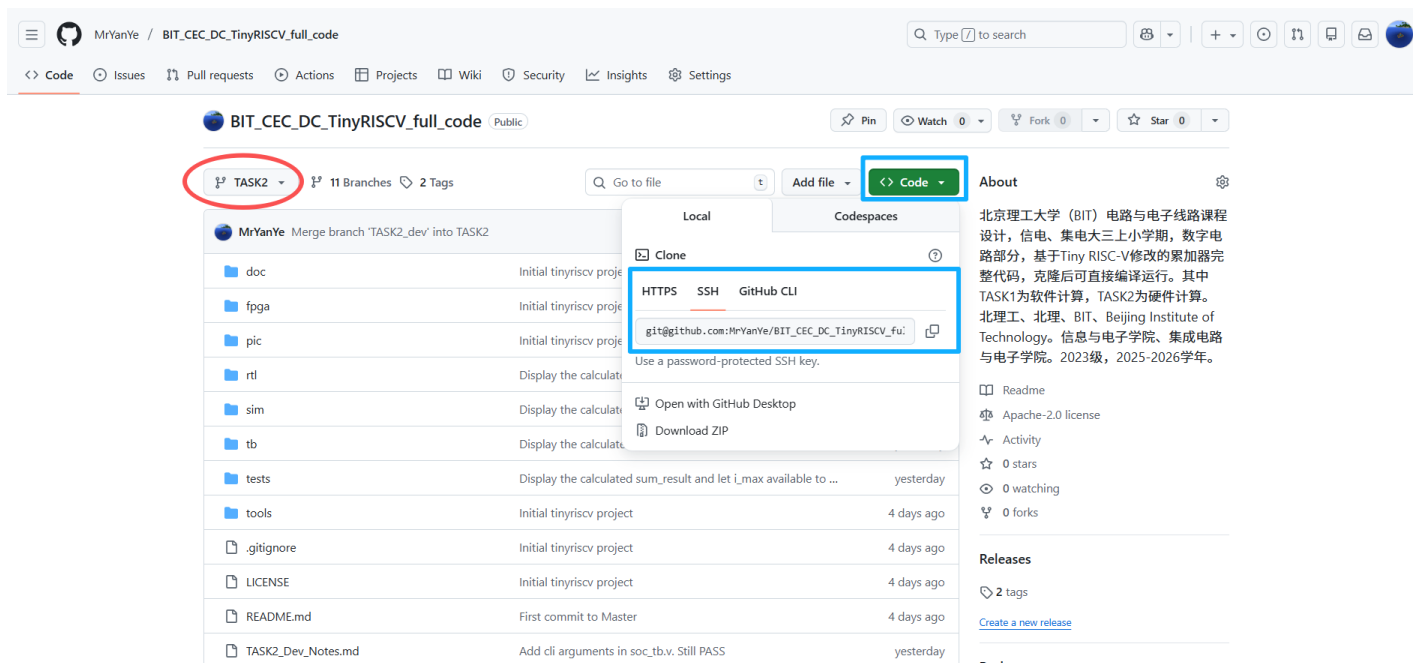
(2) 克隆我的 Github 仓库

如果您想要完整代码，请克隆下面的仓库

https://github.com/MrYanYe/BIT_CEC_DC_TinyRISCv_full_code

我已经将我的完整代码上传到上面了。

要运行此项目，要先按照 <https://gitee.com/liangkangnan/tinyriscv/tree/master> 里面的教程配置环境并安装相关工具（gnu-mcu-eclipse-riscv-none-gcc-8.2.0-2.2-20190521-0004-win64 这个文件夹我已经上传到仓库并放到正确位置了，就不用再从百度网盘下载下来替换我这个仓库里的了）。



最终复现执行用到的主要分支有两个，**Task1_teacher** 和 **TASK2**。因为数字电路 RISC-V 小学期有两个任务 TASK1 和 TASK2（虽然是任选其一），所以 Task1_teacher 和 TASK2 分支分别相当于两个任务的 main 分支。

我设置仓库呈现的默认分支是 **TASK2**，也就是我这个报告内的 **TASK2** 代码，直接克隆下来默认呈现的代码也是 **TASK2** 的。

按照 gitee 原项目部署好了环境，安装了相关工具，并按照上面方法完成代码下载后，

cmd 进入 `\tests\example\timer_int` 文件夹，依次执行执行

```
make clean
```

```
make
```

然后进入 `\sim` 文件夹，执行

```
python .\sim_new_nowave.py ..\tests\example\timer_int\timer_int.bin inst.data
```

命令行上会出现大的 **SUM PASS**，并会在下面一行显示最终的 `sum_result` 是多少，比如

```
~~~~~ SUM_TEST_PASS ~~~~~
~~~~~
~~~~~ ##### # # # # ~~~~~
~~~~~ # # # ## ## ~~~~~
~~~~~ ##### # # # ## # ~~~~~
~~~~~ # # # # # ~~~~~
~~~~~ # # # # # ~~~~~
~~~~~ ##### ##### # # ~~~~~
~~~~~
~~~~~ ##### ## ##### ##### ~~~~~
~~~~~ # # # # # # ~~~~~
~~~~~ # # # # ##### ##### ~~~~~
~~~~~ ##### ##### # # ~~~~~
~~~~~ # # # # # # ~~~~~
~~~~~ # # # ##### ##### ~~~~~
~~~~~
sum_result      =      5050
```

表示从 1 加到 100 的累加和计算正确，并显示出最终累加结果为 5050。

如果你在 `main.c` 里修改了 `i_max` 的值，也就是 `i` 从 1 加到多少的值，比如从 100 改为了 10，那么累加和应为 55，重新编译运行后命令行就会显示

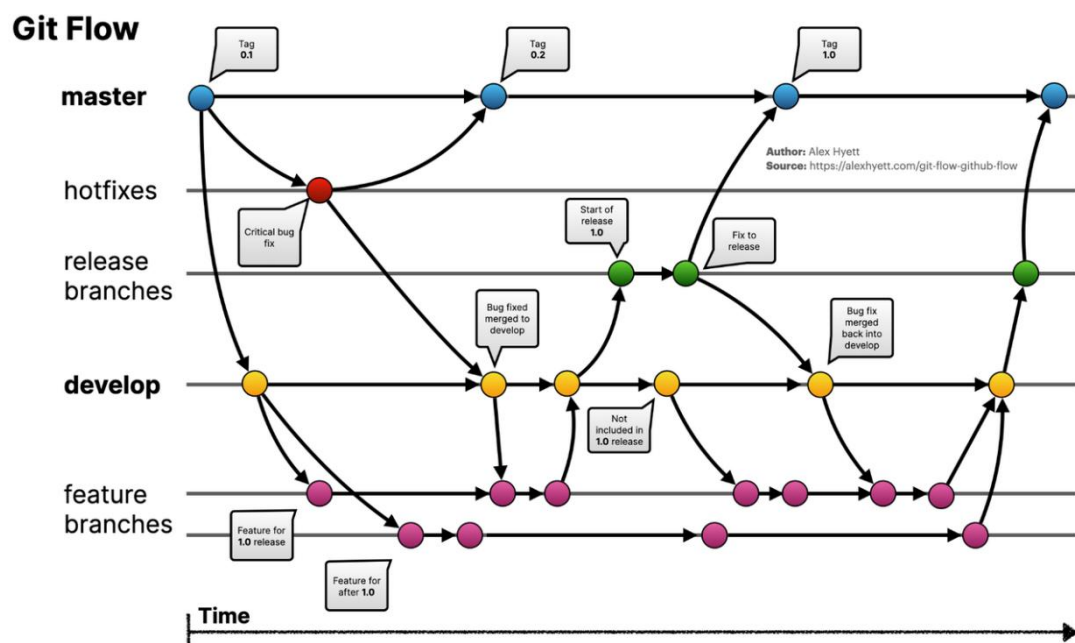
```
~~~~~ SUM_TEST_PASS ~~~~~
~~~~~
~~~~~ ##### # # # # ~~~~~
~~~~~ # # # ## ## ~~~~~
~~~~~ ##### # # # ## # ~~~~~
~~~~~ # # # # # ~~~~~
~~~~~ # # # # # ~~~~~
~~~~~ ##### ##### # # ~~~~~
~~~~~
~~~~~ ##### ## ##### ##### ~~~~~
~~~~~ # # # # # # ~~~~~
~~~~~ # # # # ##### ##### ~~~~~
~~~~~ ##### ##### # # ~~~~~
~~~~~ # # # # # # # ~~~~~
~~~~~ # # # ##### ##### ~~~~~
~~~~~
sum_result      =      55
```

可以看到同样对计算验证成功，且显示了从 1 累加到 10 的累加和为 55。

2. 代码设计及各模块介绍

2.1 整体工作流程

这次代码量虽然不多，但为了流程的清晰和可视化，以及方便回溯代码，我大致采用了 GitFlow 流程开发，也就是 main, dev, feature 等分支的框架，并给正式发布的版本打上了 tag。



最终完成后的 git branch 视图如下

```
vanZYB@DESKTOP-NKSRPTI MINGW64 /d/Project/Xilinx/CEC_Design/Digital_System_RISC_V/tinryiscv (TASK2_dev)
$ git lg
* 1c45040 (tag: v2.0.1, origin/TASK2, TASK2) Merge branch 'TASK2_dev' into TASK2
* 929b548 (HEAD -> TASK2_dev, origin/TASK2_dev) Merge branch 'feature/Add_New_Cli_args' into TASK2_dev
* dff24c5 (origin/feature/Add_New_Cli_args, feature/Add_New_Cli_args) Display the calculated sum_result and let i_max available to be set instead of the fixed 100 5050
* df24e97 Add cli arguments in soc_tb.v. Still PASS
* a791693 Update timer.v
* 0c35fca TASK2_Dev_Notes
* 0c4ad8d (tag: v2.0.0) Merge branch 'TASK2_dev' into TASK2
* 4610935 Add a judgement to sum_result sent from FPGA in main.c
* 041db17 Merge branch 'Modify_timer_v' into TASK2_dev
* 7f3318c (origin/Modify_timer_v, Modify_timer_v) Change count==2 to count==1 in main.c and PASS
* e02fd47 make successfully
* 0de385a Update timer.v 202601032341
* ff645db Add some reg variables
* c5e85a8 Add some address definition REG_I, REG_SUM_RESULT
* fe7a28f Merge branch 'C_to_REG' into TASK2_dev
* 64e8823 (origin/C_to_REG, C_to_REG) Assign TIMER0_I_MAX and TIMER0_SUM_CTRL value to their corresponding address in REG
* d1e45f5 PASS with modified timer_int/main.c
* c9a187e Add some operation to TIMER0_SUM_CTRL
* 2ed0781 PASS after the modification of timer.h
* 09db50e Merge branch 'Modify_timer_h' into TASK2_dev
* d9542a4 (origin/Modify_timer_h, Modify_timer_h) Change a Macro's name
* c932363 Add some Macros in timer.h
* 63258b9 Original project make and sim successfully
* 1af5099 Merge branch 'ComprehendTimerInt' into TASK2_dev
* 599150a (origin/ComprehendTimerInt, ComprehendTimerInt) Add some notes in timer.v
* 37c757f Add TASK2_Dev_Notes
* ed7bdec Add TASK2_Dev_Notes
* b4ccc40 (origin/TASK1_teacher, TASK1_teacher) Add some study notes in timer_int/main.c
* 9637077 Modify main.c in timer_int. int sum -> int sum = 0
* beaf1ac timer.v - addr_i [3:0] changes to [4:0]
* 4a4d0cd python sim bin
* 4cf4b62 make timer_int
* a99080b Correct main.c in timer_int
* a0f8f3d Modify timer.h
* e7467d6 Merge branch 'Modify_main_c_202601012352' into TASK1_teacher Update main.c
* d3dfcf9 (origin/Modify_main_c_202601012352, Modify_main_c_202601012352) Modify main.c 202601020000
* 3c24f8e Complete the modification to timer.v
* 2d59966 Add some variables in timer.v
* e20afdb (origin/main, main) Initial tinryiscv project
* 2ece9b7 (origin/master, master) First commit to Master
vanZYB@DESKTOP-NKSRPTI MINGW64 /d/Project/Xilinx/CEC_Design/Digital_System_RISC_V/tinryiscv (TASK2_dev)
$
```

Task1_teacher 和 **TASK2** 就分别相当于两个任务的 **main** 分支，在开发 TASK2 任务代码的过程中，把总的任务拆分成小的任务，分出不同的 **feature** 分支，比如软件 C 文件到硬件的参数传递，增加新的宏定义，硬件 verilog 实现累加，硬件再传递回软件 C 语言进行验证结果是否正确等等，然后每完成一个小功能就合并到 TASK2_dev 分支上，确保全部代码无误可运行后，merge 到 TASK2 (main) 分支上，并打标签发布正式版本。

2.2 关键代码

1. \tests\example\timer_int\main.c

```
#include <stdint.h>

#include "../include/timer.h"
#include "../include/gpio.h"
#include "../include/utils.h"

static volatile uint32_t count;

int main()
{
```

```

count = 0;
int sum_result;
sum_result = 0;    //设置初值为 0， 否则仿真时硬件会出现非稳态 xxxxx

int i_max = 100;

int correct_sum_result = 0;
int i_temp;
for ( i_temp = 0 ; i_temp <= i_max ; i_temp++ )
{
    correct_sum_result = correct_sum_result + i_temp;
}    // Should be 5050

#ifdef SIMULATION
TIMER0_REG(TIMER0_VALUE) = 500;    // 10us period
TIMER0_REG(TIMER0_CTRL) = 0x07;    // enable interrupt and start timer

TIMER0_REG(TIMER0_I_MAX) = i_max;    // 设置 i 增加到的最大值是 100
TIMER0_REG(TIMER0_SUM_CTRL) = 0x07;

while (1) {
    if (count == 1) {
        TIMER0_REG(TIMER0_CTRL) = 0x00;    // stop timer
        TIMER0_REG(TIMER0_SUM_CTRL) = 0x00;
        count = 0;
        // TODO: do something

        sum_result = TIMER0_REG(TIMER0_SUM_RESULT);
        // 读取 FPGA 算完后传过来的 5050
        if (sum_result == correct_sum_result)
            set_sum_test_pass();
        else
            set_sum_test_fail();

        break;
    }
}
#else
TIMER0_REG(TIMER0_VALUE) = 500000;    // 10ms period
TIMER0_REG(TIMER0_CTRL) = 0x07;    // enable interrupt and start timer

GPIO_REG(GPIO_CTRL) |= 0x1;    // set gpio0 output mode

while (1) {
    // 500ms
    if (count == 50) {
        count = 0;
        GPIO_REG(GPIO_DATA) ^= 0x1;    // toggle led
    }
}
#endif

return 0;
}

void timer0_irq_handler()
{

```

```

    TIMER0_REG(TIMER0_CTRL) |= (1 << 2) | (1 << 0); // clear int pending
and start timer
    TIMER0_REG(TIMER0_SUM_CTRL) |= (1 << 2) | (1 << 0);

    count++;
}

```

程序先在软件里算出 0 到 100 的累加结果 5050 作为参考值，然后在仿真模式下配置定时器周期和控制寄存器，同时把最大值 `i_max` 写入硬件寄存器，启动硬件的 `sum` 模块。定时器中断触发时会清除 `pending` 并重新启动，同时也驱动 `sum` 模块继续运行，`count` 用来记录中断次数。主循环里等待 `count` 达到 1，就停止定时器和 `sum` 模块，然后读取硬件计算出的结果寄存器，与软件算出的正确值比较，如果一致就调用 `set_sum_test_pass`，否则调用 `set_sum_test_fail`。非仿真模式下则是用定时器周期控制 GPIO 翻转，实现 LED 闪烁。整体逻辑就是通过定时器中断驱动硬件累加器运行一次完整的求和过程，再把结果和软件参考值对比来判断硬件功能是否正确。

这里我仿照原有的 `simple` 例程，首先，对于传入外设的两个数据，原先是 `TIMER0_VALUE` 和 `TIMER0_CTRL`

```

    TIMER0_REG(TIMER0_VALUE) = 500; // 10us period
    TIMER0_REG(TIMER0_CTRL) = 0x07; // enable interrupt and start
timer

```

其中 `TIMER0_VALUE` 就是原始计数器加到的最大值存放的地址，`TIMER0_CTRL` 是控制信号存放的地址。于是我仿照这两个，定义了两个新的地址 `TIMER0_I_MAX` 和 `TIMER0_SUM_CTRL`

```

    TIMER0_REG(TIMER0_I_MAX) = i_max; // 设置 i 增加到的最大值是 100
    TIMER0_REG(TIMER0_SUM_CTRL) = 0x07;

```

分别是 `i` 要累加到的最大值的地址（比如 `i` 是从 1 加到 100，那么 `i_max` 就是 100），`TIMER0_SUM_CTRL` 就是我自己设计的累加器的控制中断的信号。`main.c` 会把 `i_max` 赋给**硬件寄存器**，至于硬件怎么计算下面会说。

其中, simple 历程中给的是从 1 加到 100, TASK2 要求的也是从 1 加到 100, 但我觉得这样写死累加和太固定了, 没有什么延展性, 而且万一从 1 加到 100 有特殊情况呢, 别的累加和也没发验证。

所以我在 main.c 里面把 i_max 设置为了变量, 用户可以自己设置累加到多少, 累加到 1, 10, 50, 79, 100, 200 等等都可以, 也可以来检验代码是否健壮。并且我在 main.c 里在判断前让代码软件部分算出来累加和应该是多少, 这样用户要改变的就只有 i_max 这一个变量了。

```
int correct_sum_result = 0;
int i_temp;
for ( i_temp = 0 ; i_temp <= i_max ; i_temp++ )
{
    correct_sum_result = correct_sum_result + i_temp;
} // Should be 5050 for i=100
```

然后再判断硬件算的对不对

```
sum_result = TIMER0_REG(TIMER0_SUM_RESULT);
// 读取 FPGA 算完后传过来的 5050
if (sum_result == correct_sum_result)
    set_sum_test_pass();
else
    set_sum_test_fail();
```

timer0_irq_handler 这个定时器被开启后所执行的功能的函数也做出了一些改变, 原来是清除 TIMER0_CTRL 累加器中断挂起位, 现在加上清除 TIMER0_SUM_CTRL 的中断挂起位。

```
void timer0_irq_handler()
{
    TIMER0_REG(TIMER0_CTRL) |= (1 << 2) | (1 << 0); // clear int
    pending and start timer
    TIMER0_REG(TIMER0_SUM_CTRL) |= (1 << 2) | (1 << 0);

    count++;
}
```


当然，main.c 里最重要的部分还是添加对硬件模块内寄存器值的**读取**功能，实现硬件模块完成 sum 累加后，通过**中断触发** cpu 读取计算结果并进行**校验**。添加中断功能，实现计算完成后、CPU 回读 sum 结果并校验。

那么这个功能是在哪里实现的呢？

```
while (1) {
    if (count == 1) {
        TIMER0_REG(TIMER0_CTRL) = 0x00;    // stop timer
        TIMER0_REG(TIMER0_SUM_CTRL) = 0x00;
        count = 0;
        // TODO: do something

        sum_result = TIMER0_REG(TIMER0_SUM_RESULT);
        // 读取 FPGA 算完后传过来的 5050
        if (sum_result == correct_sum_result)
            set_sum_test_pass();
        else
            set_sum_test_fail();

        break;
    }
}
```

其中最核心的就是 `sum_result = TIMER0_REG(TIMER0_SUM_RESULT);` 这一行代码，它的意思是通过总线读取硬件计算后存放累加结果的**硬件寄存器对应地址的数据**，赋给 C 语言中的 `sum_result`，然后进行结果是否正确的判断。

`if (count == 1)` 用来判断是否中断，因为每次中断，count 都会加 1。

其中 `TIMER0_SUM_RESULT` 是地址，它也是基于 `0x20000000` 后面加上一个偏移地址得到的，和新定义的 `TIMER0_I`，`TIMER0_SUM_CTRL`，`TIMER0_I_MAX` 都在 `timer.h` 中定义，同样是地址依次加 4。

```
// Add by YanZY, 202501021845. Reference from TASK1
#define TIMER0_I (TIMER0_BASE + (12))
#define TIMER0_SUM_RESULT (TIMER0_BASE + (16))
#define TIMER0_SUM_CTRL (TIMER0_BASE + (20))
#define TIMER0_I_MAX (TIMER0_BASE + (24))
```

`set_sum_test_pass()` 是我自己**新写的命令行提示**累加和检验通过的函数，而不

是原来的 `set_test_pass`，它在 `tinyriscv_soc_tb.v` 里定义，这个后面会说到。总之，`main.c` 会判断硬件计算后传过来的累加和是否为正确的累加和 `correct_sum_result`，并在命令行打印出硬件计算出来的**累加和是多少**，并且如果检验正确就会在命令行输出大字 **SUM PASS**，反之则显示 **SUM FAIL**。

至此可以看到，`main.c` 中已经添加对硬件模块内寄存器值的读取功能，实现硬件模块完成 `sum` 累加后，通过中断触发 `cpu` 读取计算结果并进行校验，添加了中断功能，实现计算完成后、CPU 回读 `sum` 结果并校验，TASK2 的**任务（2）** 已经圆满完成了。

2. \rtl\perips\timer.v

```
// 这里位数要改成 5，要不然后面 REG_I REG_SUM 不够用了
localparam REG_CTRL = 5'd0;
localparam REG_COUNT = 5'd4;
localparam REG_VALUE = 5'd8;

localparam REG_I = 5'd12;
localparam REG_SUM_RESULT = 5'd16;
localparam REG_SUM_CTRL = 5'd20;
localparam REG_I_MAX = 5'd24;

// [0]: timer enable
// [1]: timer int enable
// [2]: timer int pending, write 1 to clear it
// addr offset: 0x00
reg[31:0] timer_ctrl;

// timer current count, read only
// addr offset: 0x04
reg[31:0] timer_count;

// timer expired value
// addr offset: 0x08
reg[31:0] timer_value;

// Add by YanZY, 202601031223
reg [31:0] timer_i = 0;           // 每次 i 的数值，变化的
reg [31:0] timer_sum_temp = 0;    // 保存每次累加后的值，不断变化的
```

```

reg [31:0] timer_sum_ctrl;
reg [31:0] timer_i_max;
reg [31:0] timer_sum_result;    // 最后累加后的结果，5050

```

这里我先在开头新定义了几个固定参数和 reg 变量。

```

localparam REG_I = 5'd12;
localparam REG_SUM_RESULT = 5'd16;
localparam REG_SUM_CTRL = 5'd20;
localparam REG_I_MAX = 5'd24;

```

这四个新定义的固定参数和前面 main.c 和 timer.h 里定义使用的四个新地址是互相对应的，之所以 timer.v 里只是后 5 位数字是因为后面 case(addr_i[4:0])会截取后 5 位地址来进行决策树操作，所以只用在开头存放偏移地址就够了，节省资源和比较时间。

新增 reg 变量中 timer_i 是每次变化的 i 的数值，会从 0，1，2，一直到 100（或者用户自己设置的 i_max）。timer_sum_temp 是保存每次累加后的值，也是不断变化的，从 0，1，3，6，10，一直到 5050。timer_sum_result 是最终硬件计算完成后存放累加和的，也就是最终的 5050。

前面说过代码是通过中断触发 cpu 读取计算结果的，那这个 zhongduan 信号从哪里来呢？

其实对原来 int_sig_o 稍微改一下就行，把 timer_ctrl 全都换为 timer_sum_ctrl。

```

// assign int_sig_o = ((timer_ctrl[2] == 1'b1) && (timer_ctrl[1] == 1'b1))? `INT_ASSERT: `INT_DEASSERT;
assign int_sig_o = ((timer_sum_ctrl[2] == 1'b1) &&
(timer_sum_ctrl[1] == 1'b1))? `INT_ASSERT: `INT_DEASSERT;

```

这样，若 timer_sum_ctrl 的第 3 位和第 2 位都为 1 则 int_sig_o 被赋值为 1，否则被赋值为 0。

那什么时候 timer_sum_ctrl 的第 3 位和第 2 位都为 1 呢？自然是从 1 加到 100 完成之后，算完了，给个输出的中断信号，接下来把工作交给软件部分，这样逻辑就通顺了。

所以，在 timer.v 的下面这里

```

        if ((timer_sum_ctrl[0] == 1'b1) && (timer_i >=
timer_i_max + 1 )) begin
            // 新增，与上面原版 if 的逻辑一致，判断 i 加到最大值则触发中
            断

            timer_sum_ctrl[0] <= 1'b0;
            timer_sum_ctrl[2] <= 1'b1;

```

当 `timer_i >= timer_i_max + 1` 时，将 `timer_sum_ctrl` 的第 3 位和第 2 位置为 1，这样就能触发中断了。至于为什么是 `timer_i_max + 1` 而不是 `timer_i_max` 呢？

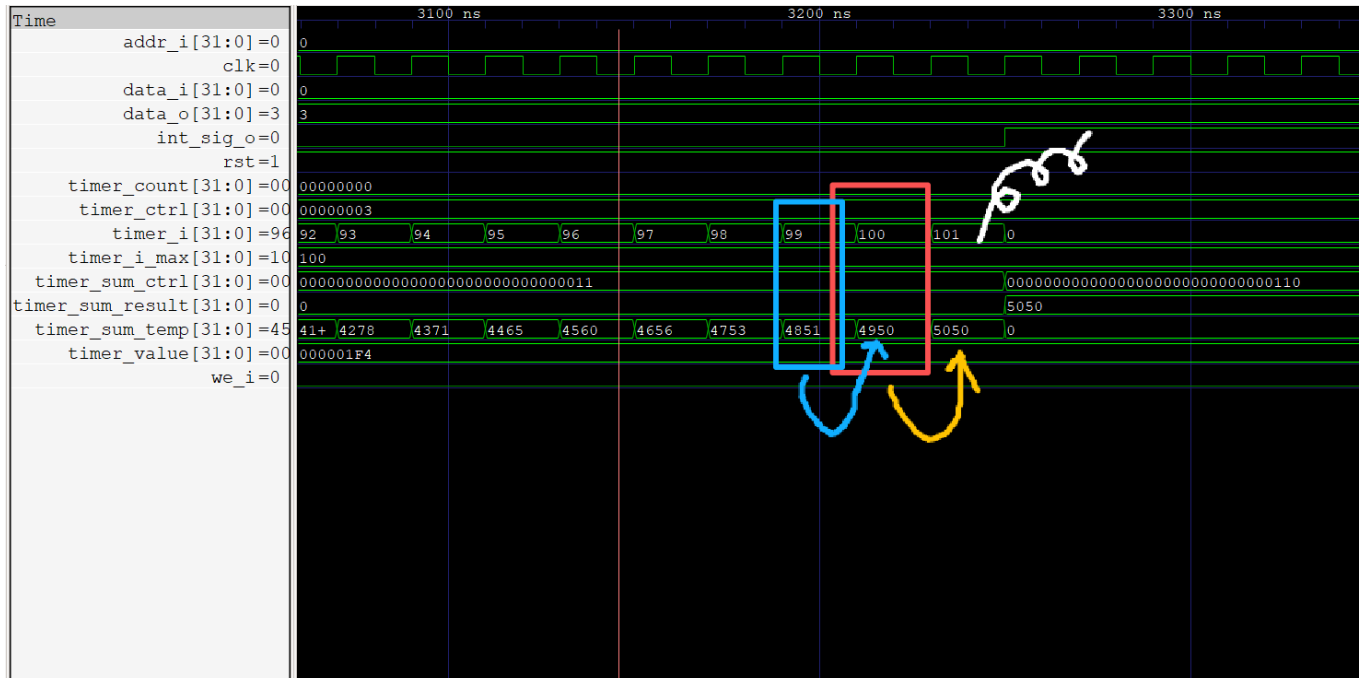
这是因为硬件每次累加时用到的是下面这样的非阻塞赋值。

```

if (timer_sum_ctrl[0] == 1'b1) begin
    timer_i <= timer_i + 1'b1;
    timer_sum_temp <= timer_sum_temp + timer_i;

```

这样 `i` 加 1 和 `sum+i` 是错开一拍的，`timer_sum_temp` 是在下一个周期显示加上这个周期的 `i` 后的值。这在波形图中也可以得到验证。



可以很明显地看到我们想要的 `sum+i` 其实是在下一个时钟周期才会在 `timer_sum_temp` 里，所以最后会导致 `i` 加到 100 还会再加一个 1，导致有一个孤零零的 101 没有参与运算，但我们正是要看 `i` 是否加到了 101 来判断累加器中的累加值是不是达到了 1 加到 100。

并且，我们只想让 timer_sum_result 在 1 到 100 算完了之后才赋值 5050，要不然直接读 timer_sum_temp 一直变太麻烦了，算完了再赋值 5050 在波形图上就特别清楚，也方便软件部分检验计算是否正确，因为只有一个值。利用 FPGA 的并行计算特性，直接新起一个 always 块，专门负责在累加算完所有数后给 timer_sum_result 赋值。

```
always @ (posedge clk) begin
    if (rst == `RstEnable) begin
        timer_sum_result <= `ZeroWord;
    end
    else begin
        if ( timer_i == timer_i_max + 1'b1 ) begin
            timer_sum_result <= timer_sum_temp;
        end
        else begin
            timer_sum_result <= timer_sum_result;
            // sum_temp 还没加到 100 的时候，timer_sum_result 保持不变
        end
    end
end
end
// 新增，对 timer_sum_result 更新，让其可以复位 0，和获得最终
timer_sum_temp 的值
```

对于从 CPU 传入的数据，延用原来的地址后 5 位截取 case 判断+data_i 赋值的方法。这里硬件计算需要的从 CPU 传来的参数其实就是两个，一个是中断控制信号 timer_sum_ctrl，一个是用户设置的累加到多少，也就是 timer_i_max。它们分别对应的地址是 REG_SUM_CTRL 和 REG_I_MAX，与 timer.h 中设置的地址偏移量相对应。

这样，我们就完成了通过指定偏移地址，从 CPU 读取传过来的配置数值的功能了。

```
case (addr_i[4:0])
    REG_SUM_CTRL: begin
        timer_sum_ctrl <= {data_i[31:3],
(timer_sum_ctrl[2] & (~data_i[2])), data_i[1:0]}; //更新 timer_sum_ctrl 的
低 2 位同时保留第 2 位的当前值，除非 data_i[2]为 1，此时第 2 位清零），避免直接写
入操作会覆盖中断挂起位
    end
```

```

REG_I_MAX: begin
    timer_i_max <= data_i;
end

```

最后，我们还要把硬件计算完成的 timer_sum_result 通过总线再重新传回给 CPU，也就是给前面设定好的地址位用 data_o 传输数据。

```

case (addr_i[4:0])
    REG_VALUE: begin
        data_o = timer_value;
    end
    REG_CTRL: begin
        data_o = timer_ctrl;
    end
    REG_COUNT: begin
        data_o = timer_count;
    end
    REG_SUM_RESULT: begin
        data_o = timer_sum_result;
    end
end

```

至此，我们就完成了将 simple 例程中的 sum 累加功能，移植到 verilog 编写的硬件模块中(timer.v)，从而将软件计算变为硬件计算，cpu 配置并启动硬件模块进行 sum 累加。成功完成了 TASK2 的任务（1）。

3. \tb\tinyriscv_soc_tb.v

首先，我们先来看 gitee 的初始项目，原始代码是通过 if (x27 == 32'b1)，也就是通用寄存器序号 x27 的值来判断是否 PASS。如果 x27 是 1 那就显示 PASS，如果 x27 是 0，那就显示 FAIL。

一共有 32 个通用寄存器 x0~x31，其中 x14 之后的寄存器都是自己定义的。

那么谁给 x27 这个通用寄存器赋值呢？

就在\tests\example\include\utils.h 这个文件里

```

#ifdef SIMULATION
#define set_test_pass() asm("li x27, 0x01")
#define set_test_fail() asm("li x27, 0x00")

```



```
if (x28 == 32'b1) begin
```

```
$display("~~~~~");  
$display("~~~~~ #####   ##   #####   ##### ~~~~~");  
$display("~~~~~ #   #   #   #   #   #   ~~~~~");  
$display("~~~~~ #   #   #   #   #####   ##### ~~~~~");  
$display("~~~~~ #####   #####   #   # ~~~~~");  
$display("~~~~~ #   #   #   #   #   #   # ~~~~~");  
$display("~~~~~ #   #   #   #####   ##### ~~~~~");  
$display("~~~~~");
```

```
end else begin
```

```
$display("~~~~~");  
$display("~~~~~#####   ##   #   #   ~~~~~");  
$display("~~~~~#   #   #   #   ~~~~~");  
$display("~~~~~#####   #   #   #   ~~~~~");  
$display("~~~~~#   #####   #   #   ~~~~~");  
$display("~~~~~#   #   #   #   #   ~~~~~");  
$display("~~~~~#   #   #   #   ##### ~~~~~");  
$display("~~~~~");  
// $display("fail testnum = %2d", x3);
```

并且我在之前故意让累加和结果为 4950，让命令行输出错误信息，然后查看得知 x14 这个通用寄存器存放的是硬件计算累加和的真实值。



所以我还在 SUM PASS 和 SUM FAIL 下面一行输出了 x14 的值，也就是硬件算出来的累加和是多少。

```
// x14 显示的是 sum_result 值 4050 或 5050
// $display("x%2d = 0x%x", r,
tinyriscv_soc_top_0.u_tinyriscv.u_regs.regs[r]);

end // if (x28 == 32'b1)

r = 14;
$display("\nsum_result
\t= %d\n", tinyriscv_soc_top_0.u_tinyriscv.u_regs.regs[r]);
```

这样一来，我就通过自定义了一个新的通用寄存器，实现了自己的命令行提示。

四、代码运行结果及波形展示

先将 main.c 中的 i_max 设为 100。

cmd 进入 \tests\example\timer_int 文件夹，依次执行执行

```
make clean
```

```
make
```

然后进入 \sim 文件夹，执行

```
python .\sim_new_nowave.py ..\tests\example\timer_int\timer_int.bin inst.da
ta
```

那么我们期望的输出累加和就是 5050

 C:\Windows\System32\cmd.exe

```
D:\Project\Xilinx\CEC Design\Digital_System_RISC_V\tinyriscv\sim>python .\sim
imer_int.bin inst.data
test running...
WARNING: ../tb/tinyriscv_soc_tb.v:548: $readmemh(inst.data): Not enough words
51.
VCD info: dumpfile tinyriscv_soc_tb.vcd opened for output.

SUM TEST PASS

#####
# # # #
# # # #
#####
# # # #
# # # #
#####
#####
# # # # #
# # # # #
#####
#####
# # # #
# # # #
#####

sum_result = 5050

../tb/tinyriscv_soc_tb.v:536: $finish called at 7030000 (1ps)

D:\Project\Xilinx\CEC Design\Digital_System_RISC_V\tinyriscv\sim>
```

可以看到确实，make 成功，生成了 bin 文件，而且在 sim 目录下的仿真的命令行看到了大字 **SUMPASS**，以及最终的 sum_result = 5050。说明软件层面验证累加和计算正确，累加和最终结果为 5050。

再将 main.c 中的 i max 设为 2, 10, 177 和 8000

i max = 2 时

C:\Windows\System32\cmd.exe

```
.: Project\Xilinx\CEC Design\Digital_System_RISC_V\tinyriscv\sim>python .\sim_
mer_int.bin inst.data
est running...
ARNING: ../tb/tinyriscv_soc_tb.v:548: $readmemh(inst.data): Not enough words
].
CD info: dumpfile tinyriscv_soc_tb.vcd opened for output.
```

```
~~~~~SUM_TEST_PASS~~~~~
~~~~~#####~~~~~
# # # # #
# # # # #
#####
# # # # #
# # # # #
#####
#####
#####
# # # # #
# # # # #
#####
#####
#####
# # # # #
# # # # #
#####
```

```
um_result      =          3

../tb/tinyriscv_soc_tb.v:536: $finish called at 5050000 (1ps)

.: Project\Xilinx\CEC Design\Digital_System_RISC_V\tinyriscv\sim>
```

i_max = 10 时

[illegible]

$i_{\max} = 177$ 时

[illegible] $i_{\max} = 8000$ 时[illegible]

可以看到，随着 `i_max` 的增加，仿真用时也越来越长，但最后都算出了正确的累加和，分别为 3, 55, 15753, 32004000。

如果我将 `timer.v` 里对累加计算完成的判断故意从

```
if ( timer_i == timer_i_max + 1'b1 ) begin
    timer_sum_result <= timer_sum_temp;
end
```

改为

```
if ( timer_i == timer_i_max + 1'b0 ) begin
    timer_sum_result <= timer_sum_temp;
end
```

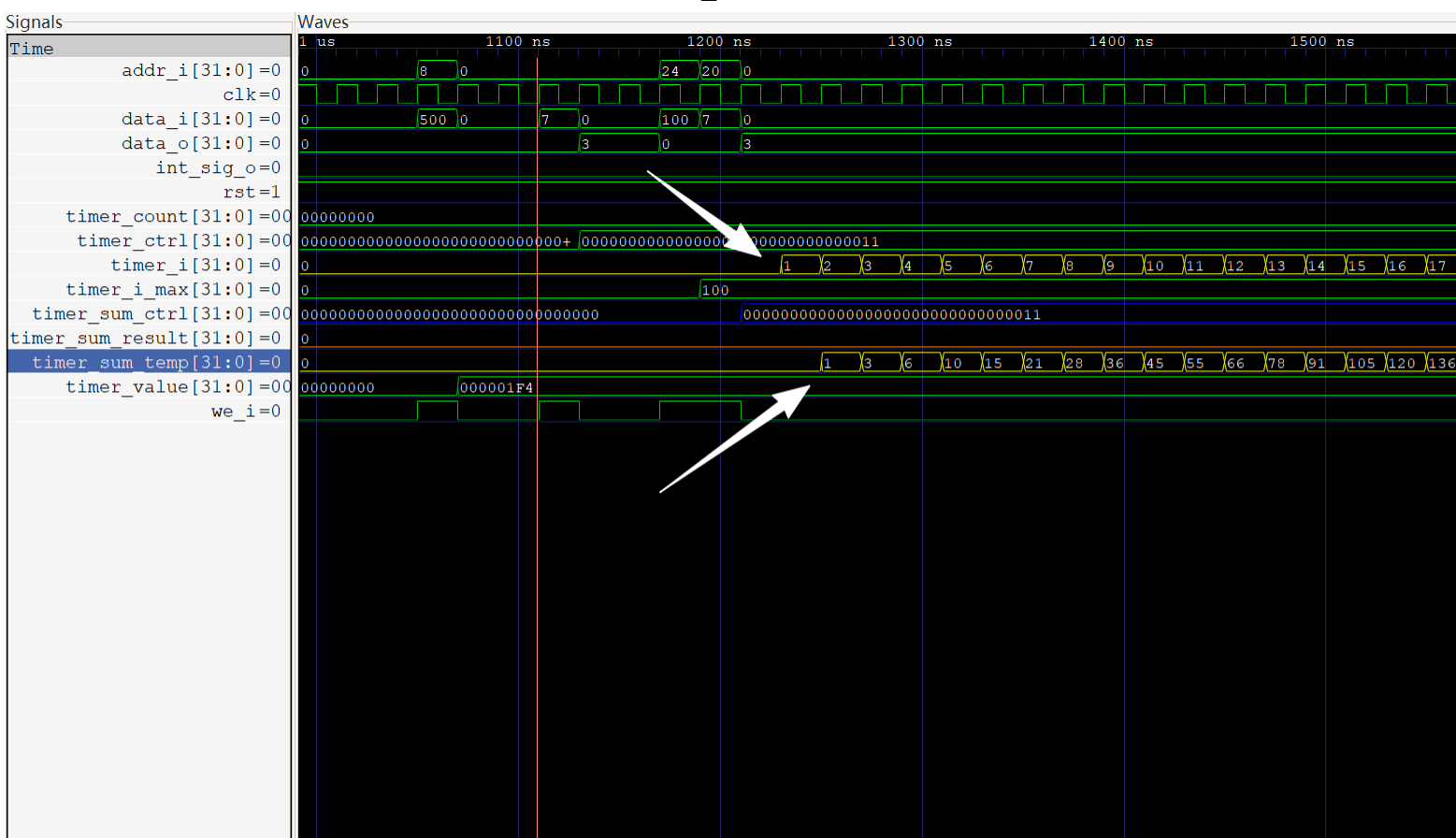
这样输出到软件层的累加和应该为 4950，因为少加了最后一个 100，所以理论上应该输出 **SUM FAIL**，sum result = 4950。实际运行结果如下：

[illegible]

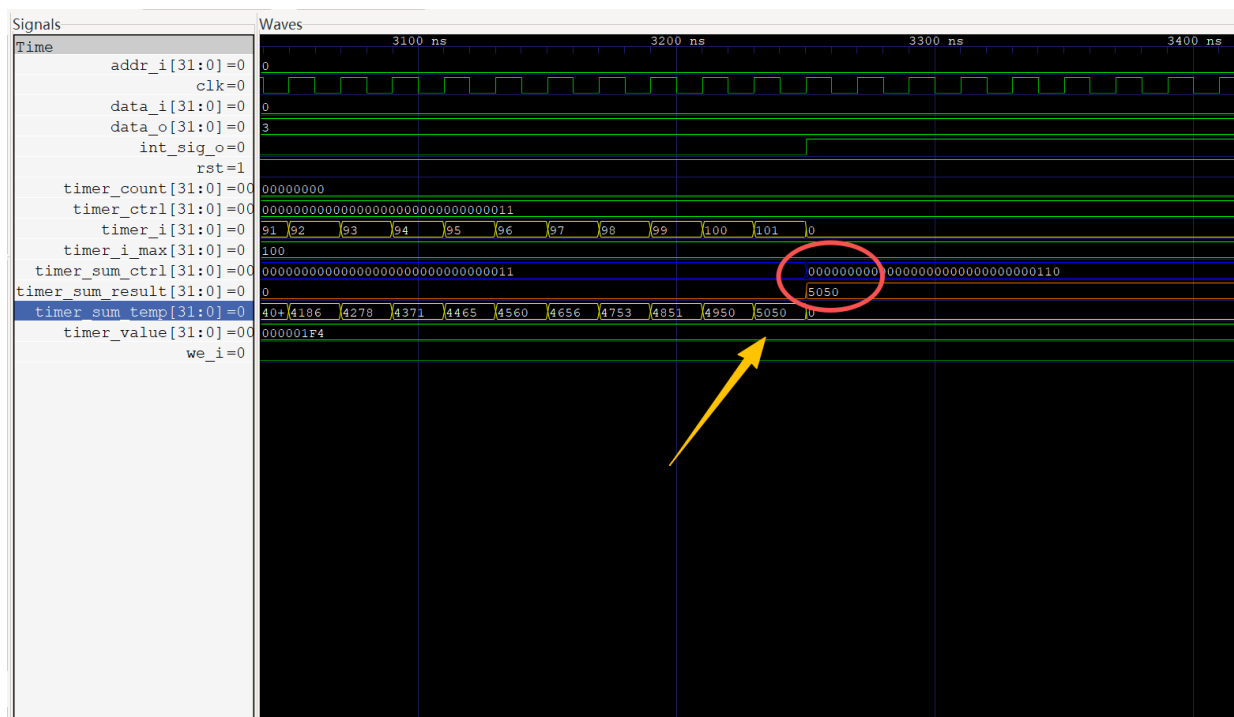
可以看到命令行果然输出了 **SUM FAIL**, sum result = 4950。

至此，我们实现了 timer 的 main.c 中添加对 sum 最终累加值的判断功能，在 cmd 的仿真命令执行后，能看到 pass/fail 提示。又成功完成了 TASK2 的任务(4)。

我们再来看输出的仿真波形，以 $i_max = 100$ 为例，也就是从 1 累加到 100。



可以看到，在 CPU 传过来 i_max 值后，`timer_i` 开始从 0 开始每个时钟周期加 1，与此同时，`timer_sum_temp` 也开始从 0，1，3，6，10，15……开始累加，符合理论数值变化的规律。而且 `timer_sum_ctrl` 也成功传来的 011 的值。



在最后加到 100 后，中断控制 `timer_sum_ctrl` 后三位变为 110，并且 `timer_sum_result` 成功被赋了 `timer_sum_temp` 的最终值 5050。

由此我们可以看到 `i` 从 1 递增至 100 时 `sum_temp` 相应的从 1 累加至 5050，可以看到 `sum` 循环累加的完整计算过程。

所以，我们也完成了 TASK2 的任务（3）。

至此，TASK2 的所有任务都圆满完成了

五、实验心得体会

最初移植 `simple` 例程里的 `sum` 功能时，我的代码并没有满足要求，硬件寄存器的值总是对不上，尤其是 `i` 和 `sum` 的变化在 GTKwave 上看起来总是差一拍，导致结果不稳定。后来我仔细分析了 `timer.v` 里非阻塞赋值的逻辑，才意识到硬件的累加过程和软件直观的循环有差别，需要在 `i_max` 的判断上加一拍才能保证最终结果正确。这个细节让我第一次真切体会到硬件时序和软件逻辑的差异，也让我明白为什么仿真工具的波形观察是必不可少的。

另一个地方是 `main.c` 里对寄存器的读取和中断触发。刚开始我只是简单地把结果读出来，但发现没有和软件算出的参考值做比较，无法验证硬件的正确性。后来我加上了 `correct_sum_result` 的计算，再和硬件返回的 `sum_result` 对比，命令行里第一次出现大字 `SUMPASS` 的时候，感觉整个流程才算真正闭环。这个过程让我认识到，硬件设计不仅仅是写 Verilog，还要和软件配合，把验证逻辑设计完整，否则实验结果没有说服力。

最后，我在 Git 分支管理上也有一些体会。虽然代码量不大，但我还是按照 GitFlow 的方式拆分成 `feature` 分支，每完成一个小功能就合并到 `dev`，再最终合并到 `main`。这样做的好处是出问题时能快速定位修改点，不会把整个项目搞乱。总体来说，这次实验让我从“能跑起来”到“跑得对”有了更深的理解，也让我在实践中体会到 CPU 架构设计的严谨性和硬件调试的细致性。